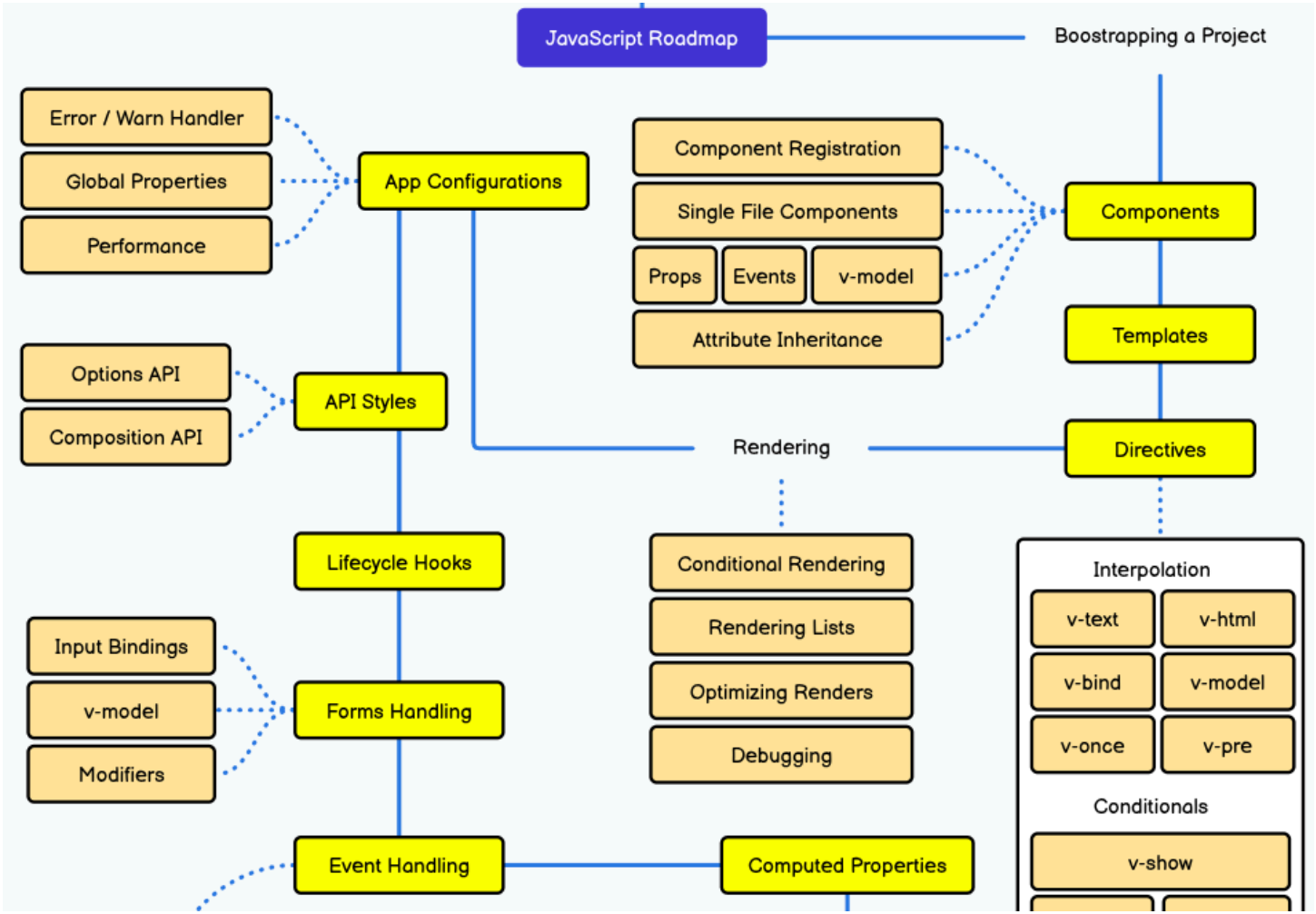
Seguimos con Vue.js



roadmap.sh/vue

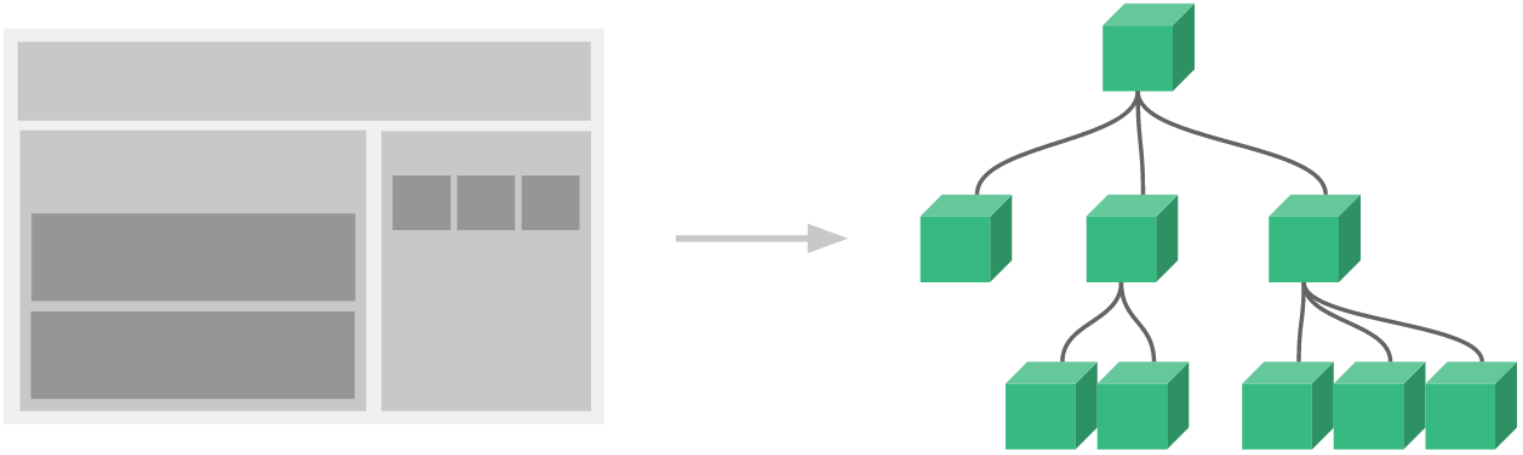
Temario

Vue.js

- Componentes: props y eventos
- Componentes single-file (SFC), Vite y create-vue
- Options API vs Composition API
- Formularios: v-model, validación y envío
- Ruteo: Vue Router
- Gestión de estado global: Pinia

Componentes Vue

- El sistema de componentes es un concepto importante.
- Nos permite construir aplicaciones grandes a partir de componentes:
  - Más pequeños
  - Reutilizables
  - Auto-contenidos



Componentes Vue

- Un componente es una pieza reutilizable con su propio template, datos y lógica. Se registra con **app.component**:

```
const app = Vue.createApp(...)
// Definimos un componente llamado todo-item
app.component('todo-item', {
  template: `<li>This is a todo</li>`
})
app.mount(...)
```

- Se puede utilizar dentro del template de otro componente:

```
  <ol><!-- Create an instance of the todo-item component -->
    <todo-item></todo-item>
  </ol>
```

Componentes Vue

- En una aplicación grande podríamos separar todo en componentes independientes.

```
<div id="app">
  <app-nav></app-nav>
  <app-view>
    <app-sidebar></app-sidebar>
    <app-content></app-content>
  </app-view>
</div>
```

Ejemplo básico componentes Vue:

- Veamos componentes-basico y componentes-multiple.

```
<div id="components-demo">
  <button-counter></button-counter>
</div>

<script>
const app = Vue.createApp({});
// Define a new component called button-counter
app.component('button-counter', {
  data: function () {
    return {
      count: 0
    }
  },
template: '<button v-on:click="count++">You clicked me {{ count }} times.</button>'
});
app.mount('#components-demo');
</script>
```

Pasando datos a un componente con props:

- Veamos componentes-props y componentes-multiple-apps

```
<div id="components-demo">
          <blog-post title="My journey with Vue"></blog-post>
          <blog-post title="Blogging with Vue"></blog-post>
          <blog-post title="Why Vue is so fun"></blog-post></div>
<script>
  const app = Vue.createApp({});
  app.component('blog-post', {
    props: ['title'],
    template: '<h3>{{ title }}</h3>'
  });
  app.mount('#components-demo');
</script>
```

- **Importante**: las props se pueden pasar únicamente hacia abajo en el árbol de componentes.

Del hijo al padre: eventos

- Un componente hijo no modifica las props: **avisa al padre emitiendo un evento** con `$emit`, y el padre lo escucha con `@`. Veamos componentes-emit.

```
<div id="components-demo">
  <p>Total: {{ total }}</p>
  <button-counter @incrementar="total++"></button-counter>
  <button-counter @incrementar="total++"></button-counter>
</div>
<script>
  const app = Vue.createApp({
    data() { return { total: 0 } }
  })
  app.component('button-counter', {
    emits: ['incrementar'],
    template: `<button @click="$emit('incrementar')">+1</button>`
  })
  app.mount('#components-demo')
</script>
```

- Regla: **props hacia abajo, eventos hacia arriba**.

Componentes single-file

- Para grandes proyectos los componentes definidos con **app.component** poseen varias **desventajas**:
  - Definiciones **globales**: cada componente debe tener un nombre único.
  - Templates como **strings**: **no poseemos syntax highlight** en el desarrollo.
  - **No tenemos soporte para CSS**.
  - **No hay etapa de construcción**: nos restringe a utilizar puramente HTML y JavaScript.
- Todo esto se soluciona utilizando componentes single-file (con extensión **.vue**) y una herramienta de construcción como **Vite**.
- Ref: https://vuejs.org/guide/scaling-up/sfc.html

Para proyectos más grandes: Vite

- Herramienta de construcción oficial para proyectos Vue.
- Mejor experiencia en desarrollo: servidor rápido y recarga en caliente (HMR).
- Mayor eficiencia en el build de producción: usa **Rolldown**.
- Multiframework: Vue, React, Svelte, etc.
- ¿Por qué Vite?: https://vite.dev/guide/why
- Guía: https://vite.dev/guide/

> **Nota**: **Vue CLI** (basado en webpack) está en **modo mantenimiento**; hoy se usa **create-vue + Vite**.

Vue + Vite: create-vue

- Scaffolding oficial para crear un proyecto Vue:

```
npm create vue@latest
```

- Pregunta qué agregar: TypeScript, **Vue Router**, **Pinia**, tests, etc.
- Genera componentes single-file con `<script setup>` (Composition API).
- Ref: https://vuejs.org/guide/scaling-up/tooling.html

Vue + Vite (cont.)

- Instalar dependencias:

```
npm install
```

- Levantar el servicio para desarrollo:

```
npm run dev
npx vite
```

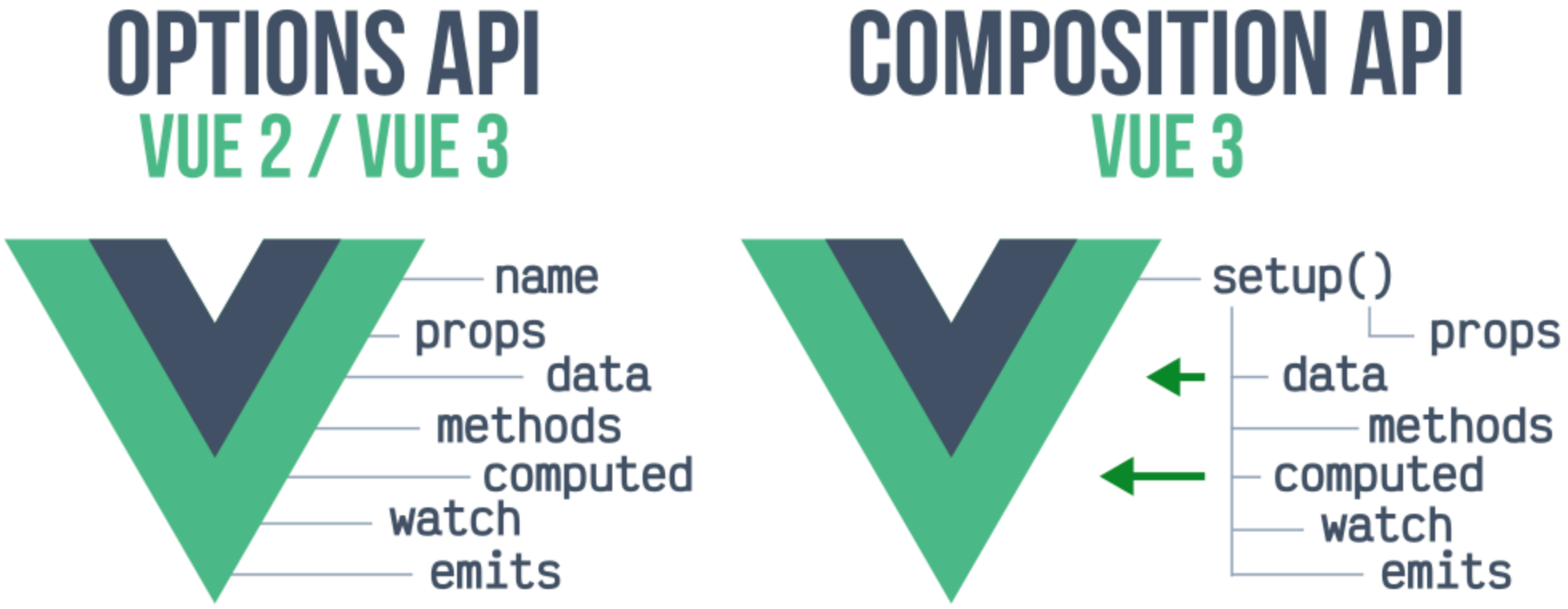
- Preview de producción:

```
npm run preview
npx vite preview
```

- Generar los archivos necesarios para producción:

```
npm run build
npx vite build
```

Options API vs Composition API



Options API

- Forma tradicional de escribir componentes (Vue 2), **sigue soportada en Vue 3**.
- Se basa en un objeto que contiene propiedades como **data**, **methods**, **computed**, **watch**, etc.
- Los datos se definen en **data**, los métodos en **methods**, y se acceden mediante **this** dentro del componente.
- Ideal para **componentes simples** y fáciles de entender.

Composition API

- Introducida en Vue 3 como una forma moderna y flexible de definir componentes.
- Se basa en funciones (**ref**, **computed**, **onMounted**, ...) que pueden agruparse y reutilizarse en **composables**.
- Permite una organización **modular** y **reutilizable** de la lógica del componente.
- Ideal para **componentes más complejos y grandes**.

El mismo componente en ambos estilos

- Mismo **template** en ambos: `<button @click="increment">{{ double }}</button>`

Options API

```
<script>
export default {
  data() { return { count: 0 } },
  computed: {
    double() { return this.count * 2 }
  },
  methods: {
    increment() { this.count++ }
  },
  mounted() {
    console.log('montado')
  }
}
</script>
```

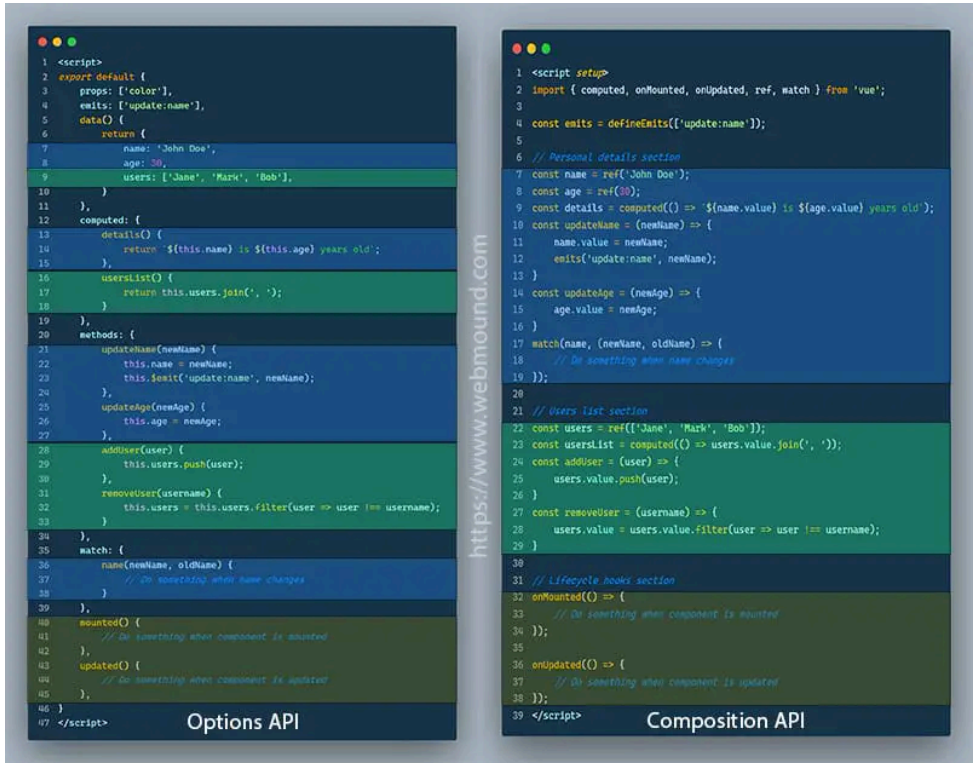
Composition API (`<script setup>`)

```
<script setup>
import { ref, computed, onMounted } from 'vue'

const count = ref(0)
const double = computed(() => count.value * 2)
function increment() { count.value++ }

onMounted(() => {
  console.log('montado')
})
</script>
```

Options API vs Composition API



- Con Composition API la lógica se **agrupa por funcionalidad** (colores), no por tipo de opción.
- Ref: https://vuejs.org/guide/extras/composition-api-faq.html

Ejemplo Hello.vue

- Veamos el fuente Hello.vue
- En **package.json** se encuentran las dependencias, instalemos con **npm install** y corramos **npm run dev**.

```
<script setup>
import { ref } from 'vue'

const greeting = ref('Hola')
</script>

<template>
  <p>{{ greeting }} World!</p>
</template>

<style scoped>
p {
  font-size: 2em;
  text-align: center;
}
</style>
```

Utilizando otros preprocesadores: pug y stylus

```
<template lang="pug">
div
  p {{ greeting }} World!
  OtherComponent
</template>
<script setup>
import { ref } from 'vue'
import OtherComponent from './OtherComponent.vue'

const greeting = ref('Hola')
</script>
<style lang="stylus" scoped>
p
  font-size 2em
  text-align center
</style>
```

- Un motor de templates: **pug** (ex jade) y un preprocesador css: **stylus**.
- Con `<script setup>` los componentes importados se pueden usar directamente en el template.

Axios + Componentes single-file

- Creamos proyecto vacío con **create-vue**:

```
npm create vue@latest
```

- Agregamos **axios**:

```
npm install axios
```

Componente ApiClient.vue: script

```
<script setup>
import { ref, onMounted } from 'vue'
import axios from 'axios'

const locations = ref([])
const errors = ref([])

// Obtiene las provincias cuando el componente se monta.
onMounted(async () => {
  try {
    const response = await axios.get(
      'https://apis.datos.gob.ar/georef/api/provincias'
    )
    // Axios parsea automáticamente las respuestas JSON.
    locations.value = response.data.provincias
  } catch (e) {
    errors.value.push(e)
  }
})
</script>
```

Componente ApiClient.vue: template

```
<template>
  <div>
    <h1>Provincias:</h1>
    <ul v-if="locations.length">
      <li v-for="location in locations" :key="location.id">
        <strong>{{ location.id }}</strong> - {{ location.nombre }}
      </li>
    </ul>

    <ul v-if="errors.length">
      <li v-for="(error, index) in errors" :key="index">
        {{ error.message }}
      </li>
    </ul>
  </div>
</template>
```

- Veamos el ejemplo en api_client.

Modificamos App.vue para incorporar el componente anterior:

```
<script setup>
import ApiClient from './components/ApiClient.vue'
</script>

<template>
  <div id="app">
    <ApiClient />
  </div>
</template>
```

- Levantemos el servicio con **npm run dev** y veamos lo construido con **npm run build**.

Formularios en Vue

- Un formulario es estado del componente: cada campo se enlaza a un dato con **v-model**.
- Agrupamos los campos en un objeto con **reactive**.
- `@submit.prevent` maneja el envío y evita que el navegador recargue la página.
- Modificadores de **v-model**:
  - `.trim`: saca los espacios del principio y del final.
  - `.number`: convierte el valor a número.
  - `.lazy`: actualiza al salir del campo (`change`), no en cada tecla.
- Ref: https://vuejs.org/guide/essentials/forms.html

Un formulario simple (Inscripcion.vue)

```
<script setup>
import { reactive } from 'vue'

const form = reactive({ nombre: '', edad: null, carrera: '', acepta: false })

function enviar() {
  console.log({ ...form })
}
</script>

<template>
  <form @submit.prevent="enviar">
    <input v-model.trim="form.nombre" placeholder="Nombre">
    <input v-model.number="form.edad" type="number" placeholder="Edad">
    <select v-model="form.carrera">
      <option disabled value="">Elegí una carrera</option>
      <option>Licenciatura en Sistemas</option>
      <option>Licenciatura en Informática</option>
    </select>
    <label><input v-model="form.acepta" type="checkbox"> Acepto los términos</label>
    <button>Enviar</button>
  </form>
</template>
```

- El checkbox da un **booleano**; el select, el valor de la opción elegida. Veamos formularios.

Validación y envío (InscripcionValidada.vue): script

```
<script setup>
import { reactive, ref, computed } from 'vue'
import axios from 'axios'

const form = reactive({ nombre: '', email: '' })
const enviando = ref(false)
const errorServidor = ref('')

// Los errores se recalculan solos cada vez que cambia el formulario.
const errores = computed(() => ({
  nombre: form.nombre ? '' : 'El nombre es obligatorio',
  email: /^\S+@\S+\.\S+$/.test(form.email) ? '' : 'Email inválido',
}))
const valido = computed(() => !errores.value.nombre && !errores.value.email)

async function enviar() {
  enviando.value = true
  errorServidor.value = ''
  try {
    await axios.post('/api/inscripciones', form)
  } catch (e) {
    errorServidor.value = e.response?.data?.error ?? e.message
  } finally {
    enviando.value = false
  }
}
</script>
```

Validación y envío (InscripcionValidada.vue): template

```
<template>
  <form @submit.prevent="enviar">
    <input v-model.trim="form.nombre" placeholder="Nombre"
           :aria-invalid="!!errores.nombre">
    <p v-if="errores.nombre" class="error">{{ errores.nombre }}</p>

    <input v-model.trim="form.email" placeholder="Email"
           :aria-invalid="!!errores.email">
    <p v-if="errores.email" class="error">{{ errores.email }}</p>

    <button :disabled="!valido || enviando">
      {{ enviando ? 'Enviando...' : 'Enviar' }}
    </button>
    <p v-if="errorServidor" class="error">{{ errorServidor }}</p>
  </form>
</template>
```

- El botón se deshabilita mientras haya errores o el envío esté en curso.
- **Importante**: validar en el cliente mejora la experiencia, pero **el servidor siempre tiene que validar**: cualquiera puede llamar a la API sin pasar por el formulario.

Estilos para los errores

- Vue no agrega estilos: la clase **error** y el atributo **aria-invalid** los ponemos nosotros en el template.
- **aria-invalid** marca el campo como inválido: lo usamos como selector CSS y además los **lectores de pantalla** lo anuncian.

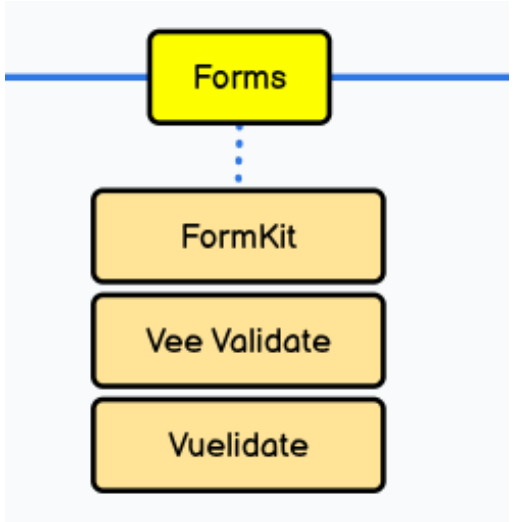
```
/* App.vue (sin scoped) o un .css global */
.error {
  color: #c0392b;
  font-size: 0.9em;
}
input[aria-invalid="true"] {
  border-color: #c0392b;
  outline-color: #c0392b;
}
```

- Con VeeValidate y Vuelidate usamos el mismo CSS; **FormKit** ya trae estos estilos en su tema.
- Ojo: este formulario muestra los errores **antes de que el usuario escriba**. Resolver eso a mano es tedioso: es lo que resuelven las librerías.

Librerías para formularios

- Con muchos formularios aparecen problemas repetidos: mostrar el error recién cuando el usuario **tocó el campo**, reglas comunes, mensajes, accesibilidad.
- Veamos el **mismo formulario** (nombre y email obligatorios) con tres librerías, en formularios:

Librería	Qué aporta
FormKit	Componentes: inputs, mensajes y estilos incluidos
VeeValidate	Composables: validación sobre **tus** inputs
Vuelidate	Reglas declaradas sobre **tu** estado



FormKit

```
npm install @formkit/vue @formkit/i18n @formkit/themes
```

main.js: registramos el plugin, en español

```
import { plugin, defaultConfig } from '@formkit/vue'
import { es } from '@formkit/i18n'
import '@formkit/themes/genesis'

createApp(App)
  .use(plugin, defaultConfig({
    locales: { es }, locale: 'es'
  }))
  .mount('#app')
```

FormKitForm.vue

```
<script setup>
function enviar(datos) {
  console.log(datos) // { nombre, email }
}
</script>

<template>
  <FormKit type="form" submit-label="Enviar"
           @submit="enviar">
    <FormKit type="text" name="nombre"
             label="Nombre" validation="required" />
    <FormKit type="email" name="email"
             label="Email" validation="required|email" />
  </FormKit>
</template>
```

- No escribimos `<input>` ni mensajes: las reglas van en **validation** y **@submit** se llama sólo si todo es válido.

VeeValidate: script

```
npm install vee-validate
```

```
<script setup>
import { useForm } from 'vee-validate'

const { defineField, handleSubmit, errors } = useForm({
  validationSchema: {
    nombre: (v) => !!v || 'El nombre es obligatorio',
    email: (v) => /^\S+@\S+\.\S+$/.test(v ?? '') || 'Email inválido',
  },
})
const [nombre] = defineField('nombre')
const [email] = defineField('email')

const enviar = handleSubmit((datos) => console.log(datos))
</script>
```

- Cada regla devuelve **true** o el mensaje de error.
- En formularios grandes, en lugar de funciones se puede usar un **esquema** de Zod o Yup: librerías que describen la forma de los datos (por ejemplo `z.string().email()`) y validan contra esa descripción.

VeeValidate: template

```
<template>
  <form @submit="enviar">
    <input v-model="nombre" placeholder="Nombre"
           :aria-invalid="!!errors.nombre">
    <p class="error">{{ errors.nombre }}</p>
    <input v-model="email" placeholder="Email"
           :aria-invalid="!!errors.email">
    <p class="error">{{ errors.email }}</p>
    <button>Enviar</button>
  </form>
</template>
```

- Seguimos escribiendo nuestros propios `<input>`: VeeValidate sólo maneja valores y errores.
- **handleSubmit** evita la recarga y llama a la función sólo si el formulario es válido.
- **errors** tiene el mensaje de cada campo, o nada si es válido.

Vuelidate: script

```
npm install @vuelidate/core @vuelidate/validators
```

```
<script setup>
import { reactive } from 'vue'
import { useVuelidate } from '@vuelidate/core'
import { required, email, helpers } from '@vuelidate/validators'

const form = reactive({ nombre: '', email: '' })
const reglas = {
  nombre: { required: helpers.withMessage('Obligatorio', required) },
  email: {
    required: helpers.withMessage('Obligatorio', required),
    email: helpers.withMessage('Email inválido', email),
  },
}
const v$ = useVuelidate(reglas, form)

async function enviar() {
  if (await v$.value.$validate()) console.log({ ...form })
}
</script>
```

Vuelidate: template

```
<template>
  <form @submit.prevent="enviar">
    <input v-model="form.nombre" placeholder="Nombre"
           :aria-invalid="v$.nombre.$error">
    <p v-if="v$.nombre.$error" class="error">
      {{ v$.nombre.$errors[0].$message }}
    </p>
    <input v-model="form.email" placeholder="Email"
           :aria-invalid="v$.email.$error">
    <p v-if="v$.email.$error" class="error">
      {{ v$.email.$errors[0].$message }}
    </p>
    <button>Enviar</button>
  </form>
</template>
```

- Los datos siguen en nuestro `reactive`: Vuelidate valida ese estado con las **reglas**.
- `$validate()` marca todos los campos como tocados y devuelve si son válidos; `$error` se activa sólo en campos tocados.
- Ojo: no tiene versiones nuevas desde **2023**.

Ruteo en Vue

- Para Vue 3 vamos a utilizar la librería vue-router y su documentación.

vue-router

- Instalación:

```
npm install vue-router
```

- Esto va a agregar **vue-router** a nuestro archivo **package.json**. También se puede elegir al crear el proyecto con **create-vue**.

main.js

- Es recomendable escribir el código de ruteo en un archivo separado **router.js** y luego agregarla a la aplicación Vue dentro del **main.js**:

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router' // Router being imported

createApp(App).use(router).mount('#app')
```

router.js

- Importamos **createRouter** del paquete **vue-router**.
- Lo exportamos al resto de la aplicación para que lo use.

```
import { createRouter, createWebHistory } from 'vue-router'

const routes = []
const router = createRouter({
  history: createWebHistory(),
  routes
})

export default router
```

Las rutas

```
const routes = [
  { path: '/', name: 'home', component: Home },
  { path: '/ruta2/:parametro', name: 'nombre-ruta2', component: Ruta2, props: true }
]
```

- **path**: El path relativo a la base de la aplicación.
- **name**: El nombre de la ruta para referenciarla en los componentes.
- **component**: El componente que va a estar en esa ruta.
- **redirect**: Una redirección.
- **alias**: Alias.
- **children**: Un arreglo con más rutas que se concatenan a la ruta padre.
- **props**: Pasa los parámetros de la ruta como props al componente.

Utilizando el router en un componente:

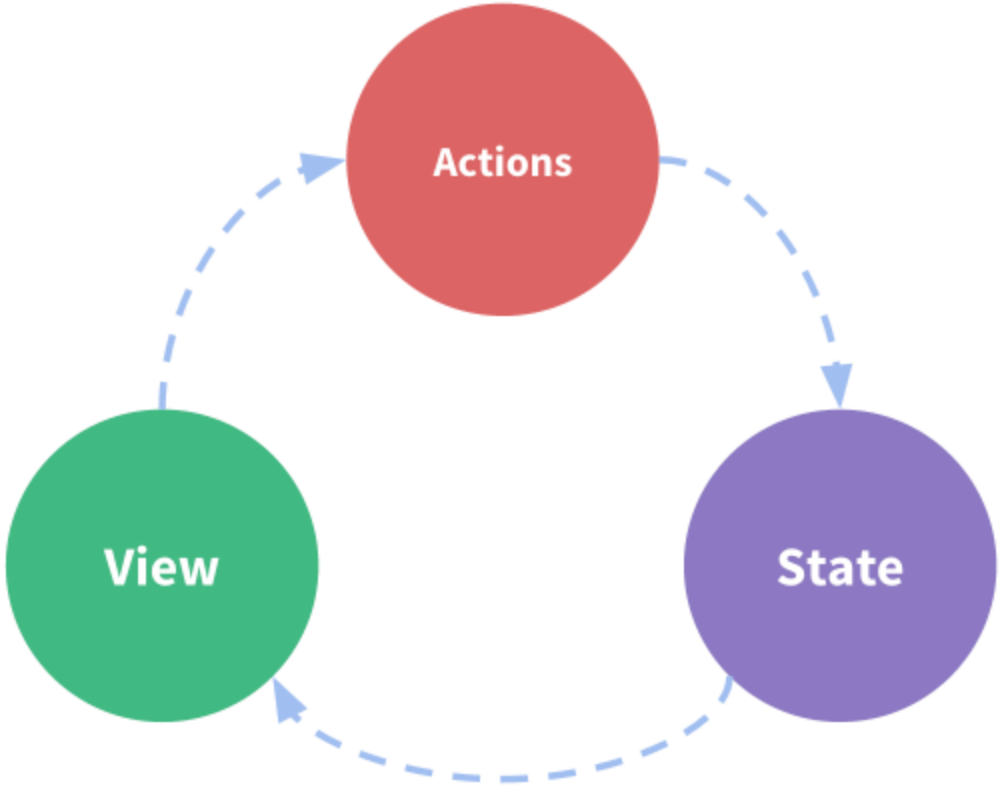
- El componente de la ruta se va a renderizar dentro del tag **router-view**.
- Para acceder a las rutas podemos utilizar un tag **a** que va a recargar la página o utilizar el componente **router-link**.

```
<template>
  <div id="app">
    <div id="nav">
      <router-link to="/">Home</router-link> |
      <router-link to="/ruta1">Ruta 1</router-link> |
      <router-link :to="{ name: 'nombre-ruta2', params: { parametro: 'Hola' } }">Ruta 2</router-link> |
      <router-link to="/about">About</router-link>
    </div>
    <router-view/>
  </div>
</template>
```

- Veamos el ejemplo en ejemplo-router.

Gestión de estado global: Pinia

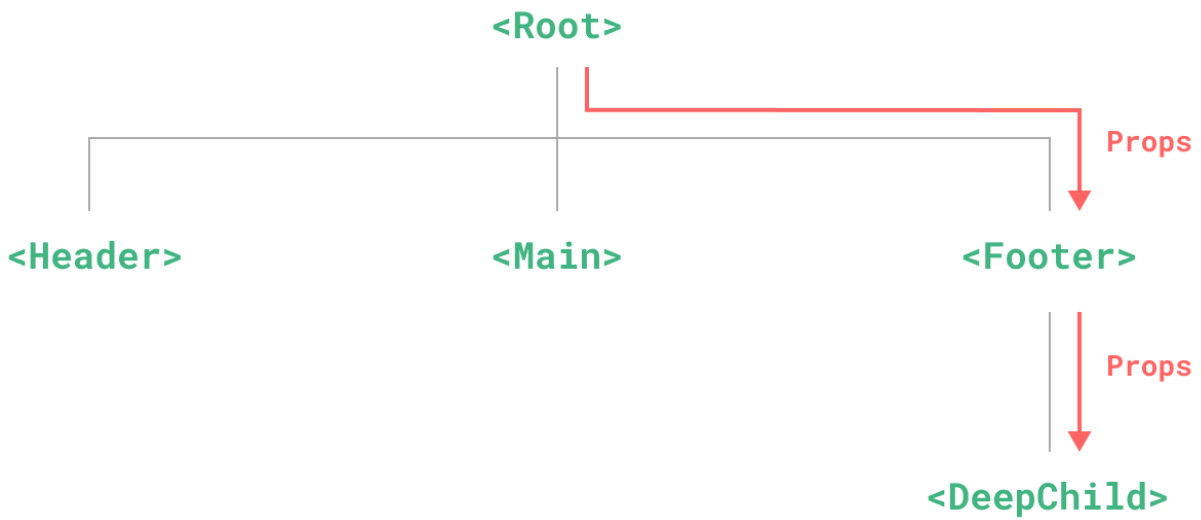
Estado local de un componente



- Cada componente maneja su propio estado: el **estado** se muestra en la **vista**, la vista dispara **acciones** y las acciones modifican el estado.

El problema: estado compartido

- En una aplicación grande varios componentes **necesitan los mismos datos** (usuario logueado, carrito, etc.).
- Con props y eventos hay que pasar los datos por componentes intermedios que no los usan (***prop drilling***):



La solución: un store con Pinia

- Un **store** es un contenedor **global** y **reactivo** del estado: cualquier componente lo lee y lo modifica directamente, sin intermediarios.
- **Pinia** es la librería **oficial recomendada**: un store por módulo, sin mutaciones, soporte para TypeScript y devtools.
- **Vuex** fue la solución anterior y hoy está en **modo mantenimiento** (puede aparecer en proyectos viejos).
- Refs: https://pinia.vuejs.org/ y comparación con Vuex

Agregando Pinia

- Instalación (o elegirlo al crear el proyecto con **create-vue**):

```
npm install pinia
```

- Lo registramos en el **main.js**:

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'

createApp(App).use(createPinia()).mount('#app')
```

Definimos un store (setup store)

- En **stores/counter.js**: **ref** = estado, **computed** = getters, funciones = actions.

```
import { ref, computed } from 'vue'
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', () => {
  const count = ref(0)
  const double = computed(() => count.value * 2)

  function increment(amount = 1) {
    count.value += amount
  }
  function decrement(amount = 1) {
    count.value -= amount
  }

  return { count, double, increment, decrement }
})
```

Usando el store en un componente (Counter.vue)

```
<script setup>
import { storeToRefs } from 'pinia'
import { useCounterStore } from '../stores/counter'

const props = defineProps({ msg: String, num: Number })

const store = useCounterStore()
const { count } = storeToRefs(store) // mantiene la reactividad
const { increment, decrement } = store // las actions, directo
</script>

<template>
  <h2>{{ msg }}</h2>
  <p>count = {{ count }}</p>
  <button @click="increment(props.num)">+{{ num }}</button>
  <button @click="decrement(props.num)">-{{ num }}</button>
</template>
```

- Veamos el ejemplo con Múltiples Contadores: todos comparten el mismo store.

La otra forma: options store

- Mismo store, con **state**, **getters** y **actions** (similar a Options API):

```
import { defineStore } from 'pinia'

export const useCounterStore = defineStore('counter', {
  state: () => ({ count: 0 }),
  getters: {
    double: (state) => state.count * 2,
  },
  actions: {
    increment(amount = 1) {
      this.count += amount
    },
  },
})
```

- Se usa igual desde los componentes. Ref: https://pinia.vuejs.org/core-concepts/

Más ejemplos: Pinia

- Veamos ejemplos:
  - Múltiples Contadores (setup store)
  - Pinia + Options API
  - Pinia + Composition API => https://github.com/piniajs/example-vue-3-vite

Para seguir leyendo: Vue.js

- Guía de Vue: https://vuejs.org/guide/introduction.html
- Composition API FAQ: https://vuejs.org/guide/extras/composition-api-faq.html
- Formularios: https://vuejs.org/guide/essentials/forms.html
- Vue Router: https://router.vuejs.org/
- Pinia: https://pinia.vuejs.org/
- Vite: https://vite.dev/guide/
- Componentes y Plugins: https://madewithvuejs.com/
- Gitlab.com usa vue: https://about.gitlab.com/blog/why-we-chose-vue/

¿Dudas?

Fin